Enhancing Reliability in LLM-Based Secure Code Generation

Mohammed F. Kharma, Mohammad Alkhanafseh, Ahmed Sabbah, David Mohaisen, *Senior Member, IEEE*

arXiv:2605.24300v1 [cs.CR] 22 May 2026

***Abstract*—Large language models (LLMs) are widely used for code generation, but their security reliability remains inconsistent across languages and prompting strategies. Existing prompt engineering improves functional correctness but rarely ensures consistent security outcomes. We introduce the *Mitigation-Aware Chain-of-Thought (MA-CoT)* framework, which embeds task-specific CWE mitigation guidance and language-aware safeguards to reduce recurring vulnerabilities in generated code. We evaluate MA-CoT across three LLMs (gpt-5, claude-4.5, gemini-2.5), three programming languages (C, Java, Python), and four prompting strategies (Vanilla, Zero-shot, CoT, MA-CoT) on a 200-task primary dataset, with external validation on LLMSecEval. Using static analysis with expert validation, MA-CoT reduces total security findings from 92 to 39 (57.6%) on the primary dataset and from 73 to 4 (94.5%) on LLMSecEval. High-severity findings (Blocker + Critical) drop from 90 to 39 (56.7%) and from 45 to 2 (95.6%), respectively. Across both datasets, MA-CoT is the only strategy that consistently improves security reliability; Zero-shot and CoT are less reliable and may increase vulnerability, especially in C. We further introduce a strict layered attribution of vulnerability drivers (language-core vs. stack layers) and show that residual risk concentrates in hardening-oriented patterns (e.g., OS- and toolchain-dependent), motivating secure-by-construction primitives alongside prompting. Code and data are available at GitHub.**

***Index Terms*—reliability, dependability, robustness, secure code generation, LLM, mitigation framework**

I. Introduction

LARGE language models (LLMs) are transforming software development by translating plain-language specifications into functionally correct code [1]–[6]. Tools such as GitHub Copilot, OpenAI Codex, and Google Gemini are now used daily by over 50% of developers, providing productivity gains of 30–50% on routine programming tasks [7], [8].

These efficiency gains are accompanied by concerns regarding the *reliability* of LLM-generated code. Empirical evidence indicates that code produced by LLMs frequently contains security vulnerabilities, which can undermine system dependability. For example, Pearce *et al.* found that Copilot generated insecure implementations in roughly 40% of security-critical scenarios, including mismanaged buffer boundaries, SQL queries, and cryptographic routines [9]. Fu *et al.* observed similar vulnerability patterns in Python and C++, which persisted even as model size and training data increased [10]. Sandoval *et al.* further showed that developers often accept insecure suggestions without sufficient review, particularly under time pressure, highlighting risks to the *reliability and robustness* of automated code generation [11].

M. Kharma, M. Alkhanafseh, A. Sabbah, and D. Mohaisen are with with the Department of Computer Science, Birzeit University, Ramallah, Palestine. D. Mohaisen is also with the Department of Computer Science, University of Central Florida, Orlando, FL 32816 USA. E-mail: mohaisen@ucf.edu (Corresponding author)

The risks of LLM-generated code extend beyond isolated errors. Asare *et al.* demonstrated that current LLMs have notable limitations in enforcing context-dependent security constraints, often generating code that appears reliable in isolation but fails to satisfy system-level security requirements [12]. Ullah *et al.* reported similar findings via benchmark tests, showing that even explicit prompts rarely enable models to consistently identify or reason about vulnerabilities [13]. These results indicate fundamental gaps in how LLMs represent and apply security knowledge during code generation, undermining the overall *reliability* of generated software.

Prompt engineering offers a method to steer model behavior without retraining. Techniques such as zero-shot, few-shot prompting, and chain-of-thought (CoT) reasoning can improve functional correctness and logical consistency [14], [15]. However, *security and reliability considerations are mostly incidental*. Broad instructions like "write secure code" are too unspecific to consistently improve reliability, and they provide limited guidance for mitigating concrete vulnerabilities [16].

Our previous work [17] introduced the Weaknesses-Aware Chain-of-Thought (WA-CoT) method, which mapped programming tasks to relevant Common Weakness Enumeration (CWE) entries and embedded them in prompt guidance to inform the LLM of potential security flows. While promising, a broader evaluation revealed several reliability-related limitations: ① *Information Overload.* Large CWE catalogs can increase, rather than reduce, failure occurrences [18]. ② *Coarse-Grained Integration.* Listing CWE identifiers and descriptions without concrete remediation steps seldom translates into secure and reliable code. ③ *Language Agnosticism.* Uniform treatment of all programming languages overlooks differences in type safety, memory management, and language-specific reliability risks. ④ *Lack of Reasoning Support Without Clear Mitigation.* Models may recognize vulnerabilities in principle but fail to apply security logic during actual code synthesis, compromising dependable code generation.

Motivated by these observations, we design MA-CoT, a prompt-based method that improves the *reliability* of LLM-generated code while remaining lightweight enough for practical deployment. MA-CoT embeds reliability-oriented reasoning directly into the code generation process through three components: mitigation-focused CWE integration, a baseline security ruleset, and language-aware prompting.

Rather than providing extensive vulnerability descriptions, MA-CoT uses **mitigation-focused CWE integration** to extract concise, actionable countermeasures from a structured CWE mitigation knowledge base. For each programming task, relevant security domains (e.g., cryptography, authentication, input validation) are identified, and targeted safeguards are incorporated into the prompt. For example, cryptographic operations trigger recommendations such as AES-256-GCM with

AEAD guarantees, Argon2id key derivation with at least 128-bit salt, and nonce generation using a cryptographically secure random number generator. This approach ensures *consistent reliability across tasks* without introducing prompt overload.

MA-CoT further includes a **baseline security ruleset**, a universal set of secure coding principles applied across all tasks regardless of language. These include strict input validation, robust error handling, safe memory management, and defensive programming. Together, they provide a foundational reliability layer that reinforces dependable code generation independently of prompt specifics.

Finally, MA-CoT employs a **language-aware prompting strategy**. Vulnerabilities manifest differently across programming languages, so the system augments a language-agnostic checklist with language-specific safeguards. In C, emphasis is placed on pointer initialization, bounds checking, and explicit memory clearing; in Python, the focus is input sanitization, safe deserialization, and proper exception handling. This alignment improves *reliability across diverse language contexts*.

A common source of confusion in language-aware secure generation is defining what constitutes a "language feature." Many weaknesses arise not from the core language specification but from the surrounding stack, including standard libraries, third-party APIs, OS interfaces, and toolchains. To avoid over-attribution and ensure actionable guidance, we adopt a strict layered framework that separates language-core mechanisms from language-adjacent drivers, allowing us to identify and mitigate residual failure modes effectively.

Research Questions. We evaluated the MA-CoT using the following research questions:

RQ1: To what extent does targeted CWE-based mitigation reduce the severity of vulnerabilities in LLM-generated code compared to standard prompting techniques?

RQ2: How do programming language characteristics influence the effectiveness of mitigation-aware prompting?

RQ3: What vulnerability patterns emerge across models, and how can they inform model-specific security?

Contributions. The contributions of this study are as follows:

1) *MA-CoT Prompt Method.* A prompting method that integrates actionable CWE mitigation guidance, a baseline security ruleset, and language-specific adaptations.
2) *Comprehensive Empirical Evaluation.* An extensive study across three state-of-the-art LLMs, three programming languages, four prompting strategies, and 200 security-relevant tasks, yielding 7,200 generated programs. Effectiveness is validated using LLMSecEval, a widely adopted benchmark for code generation [16], [19]–[21].
3) *Security Metrics and Analysis.* A detailed assessment of vulnerability density, severity distribution, and CWE category across models, languages, and prompting strategies.
4) *Open Benchmark and Reproducibility.* A publicly released benchmark including task specifications, CWE mappings, and experimental artifacts to support replication and future research on secure code generation.
5) *Layered Language-Aware Attribution and Hardening Guidance.* A conservative attribution framework that distinguishes language-core, runtime, ecosystem, OS, and toolchain factors, providing actionable, layer-specific security strengthening recommendations.

Organization. Section II reviews the related work followed by our methodology in section III, the dataset in section IV, the results in section V, discussion in section VI, threat to validity in section VII, and concluding remarks and future work in section VIII.

II. Related Work

LLM-based code generation is now used in end-to-end development workflow. However, security outcomes remain inconsistent across languages and prompt designs [9], [21]–[24]. Prior work [16], [25]–[34] has shown that prompting can steer functional behavior, yet security improvements are often inconsistent, model-dependent, and rarely driven by structured, task-specific mitigation knowledge.

Prompt engineering has evolved into a systematic discipline for shaping model behavior without parameter updates. Sahoo *et al.* [25] surveyed 41 prompt engineering techniques and highlighted the need for more application-centric prompt designs and combined prompting approaches beyond general-purpose ones. Chen *et al.* [26] systematized prompting methods and described how structural prompt changes (*e.g.*, decomposition and multi-step prompting programs) can alter model behavior. In the context of secure coding, another review of the literature [27] emphasizes that security flaws in LLM-generated code cover a wide range of vulnerability types and cannot be fully mitigated by generic warnings alone. These works motivate security-specific prompt structures that inject actionable guidance rather than relying on broad reminders.

A major line of research treats LLMs as both generators and reviewers. Tony *et al.* [16] provided a systematic investigation of prompting techniques for secure code generation and reported that Recursive Criticism and Improvement (RCI) can reduce weakness density (*e.g.*, lowered the average weakness by 77.55% compared with GPT-4), while also noting that some weakness families (e.g., command injection patterns) may remain persistent under prompting. Their analysis showed that reductions are more reliable for certain CWEs (*e.g.*, CWE-94, CWE-259, and CWE-330) than for others. More broadly, the RCI paradigm was introduced as a general strategy for improving LLM task performance via iterative critique and revision [30]. Datta *et al.* [29] studied reflection-style secure code generation at scale and quantified both improvements and potential regressions, reinforcing that multi-round repair pipelines require careful evaluation to avoid trading one weakness for another. Complementary evidence from manual reviews also suggests that repeated review cycles can increase secure output ratios when the reviewer is security-aware [28]. These results motivate MA-CoT's design choice to embed mitigation guidance directly into the reasoning process rather than relying on unconstrained self-correction.

Several approaches injected external security knowledge into generation. PromSec [34] optimizes prompts for secure code generation and reports an order-of-magnitude reduction in operational time while improving security outcomes. SecureCoder [33] maps tasks to CWE-aligned security knowledge and reports more than 65% of vulnerability mitigation

by aligning generation with CWE guidance. RESCUE [32] addresses noise in raw security documents by constructing a distilled knowledge base and hierarchical retrieval, reporting a 4.8-point average improvement. Related recipes "secure-by-instruction" proposed a systematic use of CWE documentation during training or data synthesis, but typically operate at the dataset or fine-tuning time rather than as lightweight inference-time controls [31]. In contrast, MA-CoT targets a portable inference-time mechanism: concise, task-specific CWE mitigation snippets, and language-aware safeguards.

Another direction aligns the model reasoning with the vulnerability semantics. Nong *et al.* [35] proposed vulnerability-semantics-guided prompting and reported large gains in vulnerability identification accuracy by steering multi-step reasoning toward root causes rather than surface patterns. Separately, self-planning decomposes coding intent into subproblems prior to implementation, improving the correctness of complex generation tasks [36]. These methods motivate MA-CoT's reasoning-centric design; however, MA-CoT differs from them by grounding the reasoning steps in explicit CWE mitigation actions and language-specific secure defaults.

Security behavior varies across languages and benchmarks, making cross-language evaluations essential for general claims. Recent multi-language analyses have reported uneven security and quality in LLM-generated code [37], [38]. Benchmarking has also shifted to joint functionality and security metrics. CWEval evaluates whether the generated programs are correct and secure, highlighting that passing tests does not imply security [39]. BaxBench evaluates backend generation under exploit-driven assessment and shows that security failures remain common under realistic threat models, even when functional correctness is achieved [40]. SecureAgentBench extends the evaluation to realistic vulnerability scenarios and reports that explicit security instructions alone do not reliably yield secure codes [41]. MA-CoT is positioned within this landscape as a prompt-level control designed to generalize across models, languages, and evaluation settings by integrating mitigation knowledge into the generation rationale.

III. Methodology

The proposed Mitigation-Aware Chain-of-Thought (MA-CoT) framework is designed to enhance the *reliability* of LLM-generated code. As illustrated in Figure 1, the framework operates via a multi-stage pipeline that progressively enriches prompts with security context and mitigation guidance. Each stage is intended to reduce recurring vulnerabilities and ensure consistent, dependable code generation across models, languages, and prompting strategies, thereby systematically improving the reliability of LLM-based software synthesis.

A. Pipeline Design

Stage 1: Task Analysis and Domain Classification. For a given programming task description, MA-CoT determines relevant security domains by referencing the predefined categories below. It uses LLM-based semantic analysis to classify the task and assign one or more categories as follows: ❶ *Secure Coding.* Tasks grounded in known software weaknesses based on MITRE's CWE taxonomy, e.g., injection flaws, cryptographic misuse, and insecure data handling. ❷ *Data Structures and Algorithms.* Problems focusing on core computing concepts such as array manipulation, recursion, sorting, searching, and graph traversal. ❸ *Parsing and Validation.* Tasks requiring careful input processing, constraint enforcement, and boundary checking to assess robustness and correctness. ❹ *Networking.* Scenarios emulating networked systems, e.g., authentication errors and protocol misuse. ❺ *Mathematics and Logic.* Challenges evaluating numerical reasoning, precision, and logical flow. ❻*Programming Systems and Utilities.* Utility tasks involving file handling, scripting, system configuration, and general-purpose workflows. ❼ *Concurrency and Synchronization.* Tasks targeting race conditions, thread safety, and synchronization in parallel execution tasks.

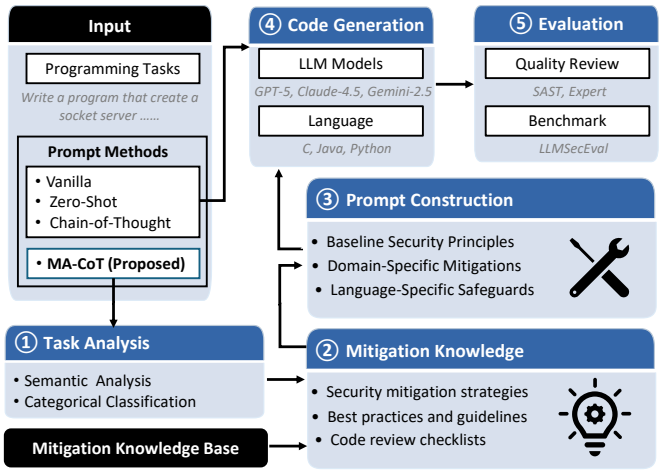


Fig. 1: Mitigation-Aware Chain-of-Thought (MA-CoT) prompting method pipeline.

Stage 2: Mitigation Retrieval. For each identified domain, the pipeline queries the code-security weakness mitigation dataset in Section IV-B to retrieve mitigation strategies for the task. This database contains mitigation entries mapped to security categories and annotated with general rules, language-agnostic guidance, code review checklists, and/or fine-tuning examples. For example, authentication-related tasks retrieve mitigation for Weak Password Requirements: "Implement strong password policies, including requirements for length, complexity, and expiration. Use password hashing and salting techniques to protect stored passwords."

Stage 3: Prompt Construction. The final prompt combines three components: ❶ the baseline security principles applicable to all code, ❷ the mitigation strategies, and ❸ the specific security checklist for the task and the programming language based on the aforementioned Stage 2. This structured approach provides the LLM with knowledge related to secure code to improve the security of LLM-generated code.

B. Task Dataset

This work builds on the dataset in section IV, created to evaluate the security and quality of LLM-generated code. In

TABLE I: Configurations: model, short name (SN), temperature (Temp), max tokens (MT), and context window (CW).

Model	SN	Temp	MT	CW
gpt-5-2025-08-07 [42]	`gpt-5`	1	32k	400k
claude-sonnet-4-5-20250929 [43]	`claude-4.5`	1	32k	200k
gemini-2.5-pro [44]	`gemini-2.5`	1	32k	1000k

addition, we developed a code weakness mitigation dataset capturing practical knowledge on how vulnerabilities should be mitigated rather than merely identified, translating it into a structure from which language models can directly learn.

C. Experimental Design

1) LLM, Language, and Prompt Selection: To examine how securely LLMs generate code under different conditions, we narrowed our evaluation to a representative set of models, programming languages, and prompt styles reflecting common practice rather than idealized setups. The choices were guided by practicality and coverage, including tools developers rely on while capturing meaningful architectural variation.

This study focuses on three LLM families: ChatGPT, Gemini, and Claude. These models are widely deployed and rank among the strongest in code generation, user base, credibility, and support. They differ in training scale, context-window limits, and safety alignment strategies, often leading to behavioral differences when handling code mixing functionality with security requirements. Table I lists the selected configurations. `Max_tokens` specifies the maximum tokens produced in completion. The temperature regulates output randomness. The CW is the largest token limit processed in a single forward pass, including prompt and generated responses, dictating how much text the LLM can handle at once.

Programming Languages. The evaluation spanned three languages: C, Java, and Python, each with distinct risks and features, allowing us to observe how LLMs adapt across languages. C is a low-level, memory-unsafe language exposing buffer overflows, use-after-free, and pointer-related vulnerabilities. Manual memory management requires careful discipline. Java is a managed language with garbage collection and strong typing, shifting vulnerabilities toward logic errors, injection flaws, and improper API usage. Python is a dynamically typed, interpreted language with an extensive standard library, where vulnerabilities involve input validation, insecure deserialization, and cryptographic misuse. Evaluating all three provides a balanced view of how models handle low- and high-level security concerns and reflects real-world software diversity.

Prompting Methods. We compare four prompting methods reflecting common LLM use: ❶ Vanilla prompting provides only the task description without security guidance as a baseline. ❷ Zero-shot prompting introduces minimal structure by encouraging secure reasoning as "Write secure code for the following task". ❸ Chain-of-Thought (CoT) extends this by requesting step-wise reasoning: (1) Understand requirements, (2) Identify security considerations, (3) Implement the solution securely, and (4) Verify correctness. Although this improves correctness, it may overlook minor security concerns. ❹ The mitigation-aware CoT prompt, proposed in this work, integrates security signals into the reasoning stage through baseline rules, task-specific mitigation, and a language checklist. Rather than correcting code after generation, it encourages models to surface potential weaknesses during reasoning.

2) Code Generation and Collection: For each combination of (LLM, language, prompting method, task), we generated one code sample, resulting in 7,200 total implementations (3 LLMs $\times$ 3 languages $\times$ 4 methods $\times$ 200 tasks). The generation used API access to the respective model endpoints with consistent parameters. The output code was extracted, compiled/syntax-checked, and stored with associated metadata (model, language, prompt method, task ID).

D. Evaluation Methodology

We used SonarQube as our main automated analysis platform [45]. It provides language-specific analyzers, and for each generated code sample we collected detected security vulnerabilities and their severity (Blocker, Critical, Major, Minor). We also mapped vulnerabilities to corresponding CWEs to identify common weakness types.

E. Benchmarking

To examine how the proposed prompting method influences code security, we used LLMSecEval (Section IV-C) as an external reference. This benchmark centers on coding scenarios where security failures are likely, rather than tasks focused only on functional output. This distinction is critical: models may produce code that compiles and behaves as expected yet includes unsafe operations, weak validation, or designs contradicting secure-coding practices. LLMSecEval captures these issues through tasks mapped to common CWE patterns, offering a clearer view of how models handle vulnerability-prone situations. The benchmark also fits the multi-language setup of our study. Because our experiments cover C, Java, and Python, a shared set of security-oriented tasks helps maintain consistency across languages with different programming paradigms and risk profiles. LLMSecEval provides consistency by framing each task to highlight the underlying security concerns rather than the surface-level syntax.

F. Language-Aware Vulnerability Attribution

A recurring source of confusion in language-aware security analysis is the term *language feature*. Many vulnerability drivers are not properties of the programming language specification itself, but of the *language stack* that developers typically use: standard libraries, primary third-party libraries, platform APIs, and compiler toolchains. To avoid over-claiming, we use a strict, layered attribution scheme that separates *language-core* mechanisms from *language-adjacent* mechanisms.

Attribution Layers. We label an attributed mechanism as *language-core* only when it follows directly from the language specification or mandatory runtime semantics (*e.g.*, C raw buffers without bounds checks, Python default-argument evaluation timing). All other drivers are classified as language-adjacent and attributed to the closest layer: (1) standard runtime/library defaults, (2) ecosystem library/framework API design, (3) platform/OS API design, (4) toolchain/optimization

TABLE II: Layered attribution used to interpret "language-aware" vulnerability drivers.

Language-core (strict): Properties required by the language spec or mandatory runtime semantics (*e.g.*, C raw memory and unchecked buffers; Python default-argument evaluation; Python string literal semantics relevant to header injection).
Standard runtime/library: Defaults and APIs shipped with the standard platform (*e.g.*, Java JCE/JSSE/JAXP configuration surfaces).
Ecosystem library/framework: Third-party or dominant frameworks and their secure-by-default posture (*e.g.*, crypto/TLS libraries, web frameworks).
Platform/OS API: System call semantics and filesystem/network primitives (*e.g.*, POSIX check-then-use patterns driving TOCTOU).
Toolchain/optimization: Compiler/linker/runtime optimizations that affect security idioms (*e.g.*, non-guaranteed memory wiping without a secure zeroization primitive).
Application security logic/policy: Input validation, password policies, and configuration choices that are not determined by the language itself.

behavior, or (5) application-level security logic/policy. This distinction is essential; e.g., TOCTOU (CWE-367) is an OS/-POSIX check-use issue that can occur in any language using the same syscalls, not a C language feature in the strict sense.

Attribution Procedure. For each SonarQube-reported weakness, we use the standardized analysis fields (language_feature, feature_category, feature_mechanism, and comparative_analysis) to assign one primary layer. When the same vulnerability could plausibly be attributed to multiple layers, we apply a conservative tie-break rule: we assign it to the *lowest* layer that is necessary to produce the risk (*e.g.*, if the risk exists because a library API allows insecure options, we assign it to the library layer rather than to the language). This results in a reproducible mapping from observed vulnerabilities to language-core versus language-adjacent mechanisms, enabling a clearer interpretation of RQ2. Table II summarizes the attribution layers and the strict criteria used to avoid mislabeling OS, library, or toolchain mechanisms as language-core features.

IV. Dataset

A. Prompt Description

This study builds on our dataset [37] developed to evaluate the functional quality and security of LLM-generated code. The dataset contains 200 language-agnostic programming tasks organized into seven functional categories. Each task includes a neutral problem statement, and generated code was evaluated using unit tests and manual and automated (tool-based) reviews, such as SonarQube. This dataset also serves as the basis for the prompt descriptions used in this study.

The reuse of this dataset offers two advantages. First, it preserves continuity with our earlier analysis, enabling us to assess whether changes in prompting or model capabilities yield measurable improvements in secure code generation. Second, because the dataset was reviewed and refined by multiple practitioners, it remains suitable for evaluating the prompt methods explored here, including Vanilla, Zero-shot, CoT, and the mitigation-aware CoT approach. We selected three LLMs and instructed them to generate code from this dataset using four prompting methods across three programming languages, producing in 7,200 samples for evaluation.

B. Mitigation-Aware Dataset

While the first dataset evaluated model outputs, the second serves a complementary purpose. It captures practical knowledge on how vulnerabilities should be mitigated rather than merely identified, translating it into a structure from which language models can learn. Each record represents a security category or specific CWE identifier, linking a problem domain to language-agnostic or language-specific mitigation guidance, review checklists, and, in some cases, fine-tuning examples.

Dataset Composition. Each entry in the Mitigation-Aware dataset contains four core components. First, *General Rules* outline baseline security practices across languages, emphasizing input validation, secret handling, cryptography, TLS configuration, and concurrency safeguards. Second, *Language-Specific Guidelines* tailor this guidance to languages such as C, C++, Python, and Java, highlighting defensive techniques like safe memory handling in C or secure resource management in Java. Third, the *Code Review Checklist* provides a concise reference to confirm that generated code meets essential safety requirements. Finally, the dataset includes a few *Fine-tuning Examples*, each in a three-part format: an instruction, an insecure input, and a secure output, illustrating correct remediation steps for models to learn. We include representative entries in the appendix. These examples show how each record integrates general mitigation principles, language-specific guidance, review checklists, and fine-tuning examples. They demonstrate the hierarchical organization of the Mitigation-Aware dataset and how security concepts are translated into practical, language-aware rules suitable for evaluation and model alignment.

Purpose and Applications. The Mitigation-Aware dataset operates at two complementary levels. First, it serves as a knowledge base combining mitigation strategies, language-aware guidance, and reasoning that supports secure-programming decisions. This structure helps trace how coding practices reduce risk, especially when a vulnerability appears correct in isolation but fails upon review. Second, it acts as a broader training corpus. Although it lacks explicit input-output pairs, its rules and rationales shape model behavior by encouraging secure design patterns. This dual role supports both model analysis and security-aware tuning. The modular structure further allows additional languages, evolving weaknesses, or CWE domains to be incorporated with minimal disruption as security expectations change.

C. LLMSecEval Benchmark Dataset

LLMSecEval [46] is widely used to evaluate language models in security-sensitive coding tasks [16], [47]–[49]. The benchmark is built from prompts derived from real C and Python examples associated with the MITRE CWE Top 25, rewritten in neutral language so the task intent remains clear without revealing the vulnerability. Each prompt is paired with a secure reference solution, enabling consistent comparison of

TABLE III: Number of vulnerabilities by programming language, model, and prompting strategy (200-task dataset).

Language	Model	Vanilla	ZeroShot	CoT	MA-CoT
C	gpt-5	17	23	24	13
	claude-4.5	19	20	27	11
	gemini-2.5	16	20	16	7
Java	gpt-5	2	3	3	2
	claude-4.5	20	13	12	1
	gemini-2.5	5	6	6	1
Python	gpt-5	3	2	2	2
	claude-4.5	7	10	4	2
	gemini-2.5	3	3	4	0

TABLE IV: Vulnerabilities by programming language, model, and prompting strategy (LLMSecEval, 150 tasks).

Language	Model	Vanilla	ZeroShot	CoT	MA-CoT
C	gpt-5	4	7	10	0
	claude-4.5	3	3	7	0
	gemini-2.5	4	5	1	1
Java	gpt-5	2	0	0	0
	claude-4.5	46	28	6	2
	gemini-2.5	5	3	3	1
Python	gpt-5	1	0	1	0
	claude-4.5	4	2	3	0
	gemini-2.5	4	6	3	0

model outputs and highlighting meaningful gaps in secure reasoning. The dataset is particularly useful for evaluating prompt methods, as many of its tasks resemble routine programming problems in which security issues often go unnoticed. By covering a range of weakness categories rather than a single CWE, LLMSecEval helps to expose recurring model errors and provides a stable baseline for measuring improvement. This makes it a practical benchmark for assessing whether a prompt strategy genuinely enhances a model's ability to avoid risky patterns and follow safer design choices.

V. Results and Discussion

We present findings from evaluating MA-CoT on three LLMs (`gpt-5`, `claude-4.5`, `gemini-2.5`), three programming languages (C, Java, Python), and two benchmark datasets, totaling 350 security-relevant programming tasks. The analysis combines automated static analysis using SonarQube with manual expert review to assess vulnerability patterns, severity distributions, and the effectiveness of mitigation-aware prompting compared to other approaches.

A. Performance on Primary Dataset

Table III presents the vulnerability counts from the 200-task dataset across all evaluated configurations. The results indicate substantial variation in baseline security behavior and the way different models respond to prompting interventions.

Several patterns emerged from this evaluation. First, MA-CoT consistently produced the fewest vulnerabilities in seven of the nine configurations. The exceptions occur with `gpt-5` in Java, where Vanilla and MA-CoT both generate two vulnerabilities, and with `gpt-5` in Python, where ZeroShot, CoT, and MA-CoT all generate two vulnerabilities. Second, the ZeroShot and CoT prompting methods frequently fail to improve security over the Vanilla baseline, except in Java and partially in Python for `claude-4.5`. In C, ZeroShot and CoT increase the vulnerability counts for all three models, with CoT generating 27 issues for `claude-4.5` compared to 19 in Vanilla. This surprising result suggests that encouraging step-by-step reasoning without concrete security knowledge can degrade code security.

Takeaway 1: MA-CoT delivers the most consistent security gains, whereas ZeroShot and CoT often fail to help and can even increase vulnerabilities, particularly in C, suggesting that unguided reasoning may unintentionally weaken code security.

The magnitude of improvement in MA-CoT varied by language. C shows clear gains, reducing vulnerabilities from 52 (vanilla) to 31 (MA-CoT), a 40% decrease. Java shows more dramatic results, dropping from 27 to 4 vulnerabilities (85% reduction). Python achieved similar success, with a 69% reduction from 13 to 4 issues. These differences reflect how cryptographic algorithms, memory management, and certificate validation affect baseline code security.

`claude-4.5` shows notable behavior. It produces the highest Vanilla vulnerability count (46 across all languages), but achieves the largest relative improvement under MA-CoT, reducing issues to 14 in total. This 70% reduction indicates a strong instruction-following capability paired with weak security priors. The model appears to lack internal safeguards that would prevent vulnerabilities in unconstrained generation but can effectively apply explicit mitigation guidance when provided with such safeguards.

Takeaway 2: `claude-4.5` demonstrates limited built-in security safeguards, yet it shows strong responsiveness to structured mitigation prompts, resulting in a marked reduction in vulnerability cases under guided conditions.

B. Benchmarking using LLMSecEval

Table IV presents the results of the LLMSecEval benchmark, which contains 150 security-focused tasks designed independently from our primary dataset. This external validation tests whether MA-CoT generalizes beyond the specific characteristics of the evaluation dataset.

The LLMSecEval results validated the patterns observed in our primary dataset. MA-CoT achieved complete vulnerability elimination in six of the nine configurations. When vulnerabilities persist, they remain minimal (one or two instances) and outperform all other prompting approaches. The method is effective even for tasks explicitly constructed to expose weaknesses in automated code generation.

A comparison of the two datasets shows both consistency and instructive differences. The relative ranking of the prompting methods remained stable: MA-CoT consistently outperformed the alternatives, while ZeroShot, CoT, and Vanilla showed no consistency in their security quality. However, the absolute vulnerability counts differed substantially. LLMSecEval produces a higher vulnerability ratio using Vanilla in Java than our dataset (53 versus 27 in our dataset), but lower C counts (11 versus 52). These variations reflect differences in tasks and programming languages rather than evaluation methods. LLMSecEval emphasizes web security patterns common in Java applications, while our dataset includes more system programming scenarios typical of C development.

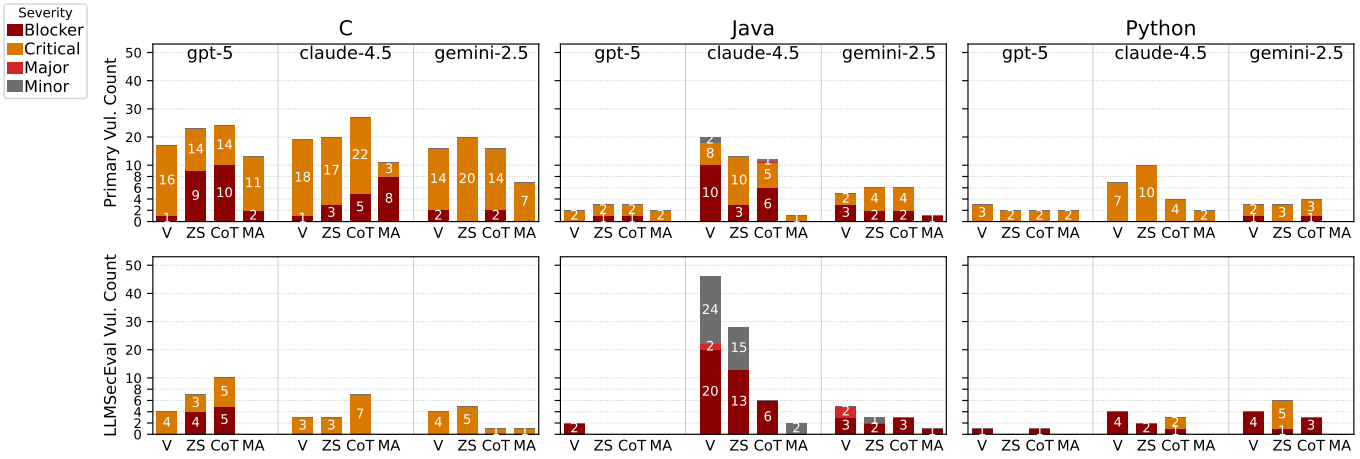


Fig. 2: Vulnerability severity counts across two datasets. Panels are organized by dataset (rows) and language (columns). Within each panel, bars are grouped by LLM and ordered by prompting method (V=Vanilla, ZS=Zero-shot, CoT=Chain-of-Thought, MA=MA-CoT). Stacks show severity counts (Blocker, Critical, Major, Minor).

Takeaway 3: MA-CoT is consistently the most secure prompting method: it removes vulnerabilities in most settings, and when issues remain, they are few and fewer than all alternatives. Across both datasets, MA-CoT maintained the top rank, while ZeroShot, CoT, and Vanilla were unstable. Differences in raw vulnerability counts stem mainly from task types and language focus, not evaluation flaws.

The consistent and counterintuitive CoT behavior across both datasets warrants closer examination. In C, CoT yields higher vulnerability rates than Vanilla in five model-dataset pairs. When given general reasoning instructions without explicit security guidance, models fail to avoid vulnerabilities that are sometimes prevented in Vanilla and ZeroShot settings.

C. Vulnerability Severity Analysis

Severity reflects the likely impact of the identified vulnerabilities and provides more insight than raw counts alone. SonarQube categorizes each issue into one of four levels (Blocker, Critical, Major, Minor). We present severity distributions by language, LLM, and prompting strategy in Figure 2.

Primary Dataset. The findings were concentrated almost exclusively at the two highest severity levels. In the Vanilla setting, we observed 92 total findings, of which 90 were Blocker or Critical (18 Blocker, 72 Critical, 2 Minor). With MA-CoT, the total decreases to 39 findings, all of them high-severity (11 Blocker, 28 Critical), corresponding to a 56.7% reduction in Blocker and Critical issues (from 90 to 39). This improvement is consistent across languages: in Java, high-severity findings fall from 25 (13 Blocker, 12 Critical) to 4 (1 Blocker, 3 Critical), and in Python from 13 (1 Blocker, 12 Critical) to 4 (0 Blocker, 4 Critical). C shows a category shift under MA-CoT: Critical findings drop from 48 to 21, while Blocker increases from 4 to 10, the combined high-severity total (Blocker plus Critical) still decreases from 52 to 31. The Blocker increase is dominated by `c:S5798` rule (*"`memset` should not be used to delete sensitive data"*). This rule flags a specific insecure-erasure idiom, using `memset()` to scrub a buffer immediately before releasing it, because the compiler may optimize the write operation. In other words, MA-CoT reduces the overall high-severity burden in C, but it also introduces more *attempted* buffer-sanitization steps that must be implemented with a guaranteed-wipe primitive to be correct. Therefore, we treat `c:S5798` rule as an implementation-level hardening issue rather than evidence of weaker security, and document the remediation in the appendix.

Benchmark Dataset (LLMSecEval). The benchmark shows a broader severity distribution than the primary dataset, including non-zero Major and Minor findings. Under Vanilla, we recorded 73 findings in total (34 Blocker, 11 Critical, 4 Major, 24 Minor), of which 45 were high-severity (Blocker+Critical). MA-CoT reduces this to 4 findings (1 Blocker, 1 Critical, 0 Major, 2 Minor), reducing high-severity by 95.6% (45 to 2) and total findings by 94.5% (73 to 4). The same pattern holds across languages: in C, Critical decreases from 11 to 1; in Java, Blocker decreases from 25 to 1, and Major is eliminated (4 to 0); and in Python, Blocker decreases from 9 to 0, with no remaining findings under MA-CoT.

Cross-Dataset Interpretation. Figure 2 provides four comparisons: (1) severity composition for a fixed configuration (stack segments), (2) differences between prompting methods within each LLM (bar order per group), (3) variation across languages (C, Java, Python), and (4) stability across datasets (Primary on top, LLMSecEval on bottom). In the primary dataset, findings concentrate in Blocker and Critical, whereas LLMSecEval includes Major and Minor findings. MA-CoT reduces Blocker and Critical in both datasets and also lowers low-severity findings in LLMSecEval. We report per-category counts rather than totals to avoid masking shifts between severity levels and to keep improvements visible per language, model, and prompting method.

Takeaway 4: MA-CoT lowers high-severity vulnerabilities (Blocker and Critical) across both datasets: by 56.7% in the primary dataset (90 to 39) and by 95.6% in LLMSecEval (45 to 2). On LLMSecEval, it also reduces total findings by 94.5% (73 to 4), indicating robustness under dataset shift.

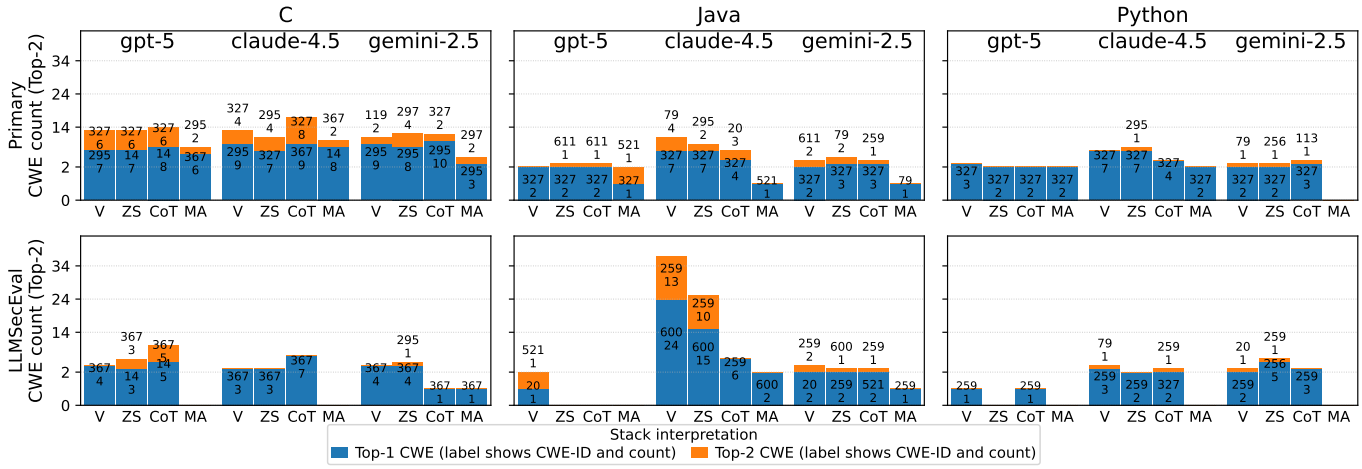


Fig. 3: Common Weakness Enumeration (CWE) patterns across datasets, languages, LLMs, and prompting methods. Rows correspond to datasets (top: Primary, bottom: LLMSecEval). The columns correspond to the languages (C, Java, and Python). Within each panel, bars are grouped by LLM and ordered by prompting method (V=Vanilla, ZS=Zero-shot, CoT=Chain-of-Thought, MA=MA-CoT).

Takeaway 5: In C of the primary dataset, MA-CoT reduces Blocker and Critical (52 to 31) while increasing Blocker due to strict secure-erasure enforcement, dominated by `c:S5798` rule. This reflects increased use of hardening patterns implemented with a non-guaranteed primitive (plain `memset`), not weaker security: the net high-severity burden decreases, and remaining issues are concentrated in a small set of cases.

D. Common Weakness Enumeration Patterns

Severity counts quantify impact but do not explain recurring weakness classes. We therefore analyzed the Common Weakness Enumeration (CWE) categories reported by SonarQube to characterize recurring vulnerability types by dataset, language, LLM, and prompting method. Figure 3 summarizes the two most frequent CWE categories per configuration, while Table V reports the top five CWEs, providing a fine-grained view of which classes MA-CoT removes and which remain.

Primary Dataset. In C, baseline prompting is marked by certificate-validation and cryptography issues, mainly CWE-295 (Improper Certificate Validation) and CWE-327 (Broken or Risky Cryptographic Algorithm). Aggregated across LLMs, these remain frequent under Vanilla, Zero-shot, and CoT (CWE-295: 25, 14, 18; CWE-327: 12, 15, 16). Under MA-CoT, both drop sharply (CWE-295=6; CWE-327=2). Residual findings concentrate on CWE-367 (TOCTOU Race Condition) and CWE-14 (Compiler Removal of Code to Clear Buffers), linked to specific idioms: TOCTOU arises from check-then-use flows, while CWE-14 is triggered when a wipe such as `memset` is not guaranteed under compiler optimization (appendix).

The appearance of CWE-367 and CWE-14 under MA-CoT requires careful interpretation. In our outputs, these reflect security hardening that triggers stricter static analysis. For example, file-existence checks before opening can create a time-of-check time-of-use window (CWE-367). Similarly, explicit buffer scrubbing with `memset` may be flagged because the compiler can optimize the write, motivating guaranteed-wipe primitives (appendix B). We therefore treat these residual findings as implementation-level hardening issues rather than baseline cryptographic or certificate-validation failures.

TABLE V: Top-5 CWE categories per language, LLM, and prompting method across datasets. Each cell lists up to five categories as CWE-ID (count), without the `CWE-` prefix.

Lang.	LLM	Method	Primary (Top-5 CWEs)	LLMSecEval (Top-5 CWEs)
C	GPT-5	V	295(7), 327(6), 367(2), 119(1), 297(1)	367(4)
		ZS	14(7), 327(6), 367(5), 119(2), 295(2)	14(3), 367(3), 119(1)
		CoT	14(8), 327(6), 295(4), 367(3), 119(2)	14(5), 367(5)
		MA	367(6), 295(2), 327(2), 14(1), 119(1)	–
	Claude-4.5	V	295(9), 327(4), 297(3), 367(2), 119(1)	367(3)
		ZS	327(7), 295(4), 367(4), 119(3), 297(2)	367(3)
		CoT	367(9), 327(8), 14(5), 295(4), 297(1)	367(7)
		MA	14(8), 367(2), 295(1)	–
	Gemini-2.5	V	295(9), 119(2), 297(2), 327(2), 367(1)	367(4)
		ZS	295(8), 297(4), 367(4), 120(2), 327(2)	367(4), 295(1)
		CoT	295(10), 327(2), 119(1), 297(1), 367(1)	367(1)
		MA	295(3), 297(2), 367(2)	367(1)
Java	GPT-5	V	327(2)	20(1), 521(1)
		ZS	327(2), 611(1)	–
		CoT	327(2), 611(1)	–
		MA	327(1), 521(1)	–
	Claude-4.5	V	327(7), 79(4), 20(3), 600(2), 611(2)	600(24), 259(13), 20(9)
		ZS	327(7), 79(2), 295(2), 521(2)	600(15), 259(10), 79(2), 521(1)
		CoT	327(4), 20(3), 259(2), 79(1), 521(1)	259(6)
		MA	521(1)	600(2)
	Gemini-2.5	V	327(2), 611(2), 521(1)	20(2), 259(2), 521(1)
		ZS	327(3), 79(2), 521(1)	259(2), 600(1)
		CoT	327(3), 79(1), 259(1), 323(1)	521(2), 259(1)
		MA	79(1)	259(1)
Python	GPT-5	V	327(3)	259(1)
		ZS	327(2)	–
		CoT	327(2)	259(1)
		MA	327(2)	–
	Claude-4.5	V	327(7)	259(3), 79(1)
		ZS	327(7), 295(1), 297(1), 521(1)	259(2)
		CoT	327(4)	327(2), 259(1)
	Gemini-2.5	V	327(2), 79(1)	259(2), 20(1), 79(1)
		ZS	327(2), 256(1)	256(5), 259(1)
		CoT	327(3), 113(1)	259(3)
		MA	–	–

Java shows a different profile: CWE-327 persists under baseline alongside web and API-misuse patterns such as CWE-611 (Improper Restriction of XML External Entity Reference) and CWE-79 (Cross-site Scripting). Under MA-CoT, CWE-611 is absent in the primary dataset, and CWE-79 is strongly reduced, indicating that explicit configuration-level guidance is

most effective when security depends on correct API defaults rather than syntactic correctness alone.

Python shows the most concentrated weaknesses in the primary dataset, with CWE-327 leading in multiple baseline configurations. This aligns with Python's memory safety by design, where security failures cluster around cryptographic misuse and unsafe parameter choices. Under MA-CoT, Python findings are minimized and eliminated in some settings.

Cryptographic weaknesses (CWE-327) appear in all three languages, suggesting that secure cryptography remains challenging, regardless of the language design. In the primary dataset, the reductions in CWE-327 under MA-CoT were substantial when measured against the Vanilla baseline: in C, 12 to 2 (83%); in Java, 11 to 1 (91%); and in Python, 12 to 4 (67%). These reductions are consistent with MA-CoT prompts that constrain algorithm choices and parameters toward modern, safe defaults. The appendix provides a task-level qualitative example for `claude-4.5`, showing this shift from ECB/CBC-style implementations under baseline prompts to AEAD-based designs under MA-CoT for the same encryption task.

Takeaway 6: MA-CoT suppresses recurring high-impact CWE classes across languages and LLMs, indicating a **consistent reduction** in systematic security failure modes rather than only a drop in total results.

LLMSecEval Benchmark. The benchmark preserves the ranking observed in the primary dataset while altering the prevalent CWE mix for some languages. In C, findings are mainly race and timing issues (CWE-367), and MA-CoT removes them in most configurations, leaving one residual case. In Java, the benchmark exposes uncaught exceptions and authentication and credential-management weaknesses (*i.e.*, CWE-600 and CWE-259), which MA-CoT substantially reduces compared to baseline prompts. In Python, findings also concentrate on few categories, and MA-CoT eliminates them across configurations. Overall, MA-CoT not only lowers total counts but filters high-risk, frequent CWE categories, leaving a smaller set tied to specific implementations and easier to mitigate with focused hardening.

E. Model-Specific Security Characteristics

Table VI compares the three models by aggregating vulnerabilities in C, Java, and Python, reported separately for the Primary (200 tasks) and LLMSecEval (150 tasks) datasets. Because the datasets differ in size and composition, we interpret cross-dataset differences qualitatively and focus on within-dataset changes across prompting methods. Across both datasets, MA-CoT was the only prompting strategy that consistently reduced vulnerability counts for every model. In contrast, generic security phrasing (Zero-shot) and basic reasoning (CoT) are not dependable: on the primary dataset, they increase findings for all models relative to Vanilla, and on LLMSecEval, they remain less effective than MA-CoT.

Claude. `claude-4.5` has the weakest baseline in the primary dataset (Vanilla=46) but the strongest improvement under MA-CoT (14), a 70% reduction. The same pattern appears in LLMSecEval (53 to 2, 96%). This indicates high responsiveness to mitigation-aware constraints: when guidance specifies safeguards, the model produces measurably safer code.

TABLE VI: Total vulnerabilities by model aggregated across languages. Prompting methods: V=Vanilla, ZS=Zero-shot, CoT=Chain-of-Thought, MA=MA-CoT.

	Primary Dataset				LLMSecEval Dataset			
Model	V	ZS	CoT	MA	V	ZS	CoT	MA
`gpt-5`	22	28	29	17	7	7	11	0
`claude-4.5`	46	43	43	14	53	33	16	2
`gemini-2.5`	24	29	26	8	13	14	7	2

Gemini. `gemini-2.5` achieves the lowest MA-CoT total in the primary dataset (8), improving from Vanilla=24 (67%). In LLMSecEval, MA-CoT reduced the findings from 13 to 2 (85%). In particular, `gemini-2.5` also shows less degradation under CoT in the reference dataset (7) than in the primary dataset (26), suggesting that its performance is more sensitive to task composition than to model capacity alone.

ChatGPT. `gpt-5` is comparatively stable but less responsive in the primary dataset: Vanilla, Zero-shot, and CoT are close (22, 28, 29), and MA-CoT reduces to 17 (23%). In LLMSecEval, however, MA-CoT eliminates all detected findings. This suggests that mitigation-aware prompting effectiveness depends on the task vulnerability mix.

These results support two conclusions: First, model selection and prompt design should be considered jointly; a weaker baseline model can still be effective when mitigation-aware prompting is consistently applied. Second, relying on generic "be secure" or reasoning prompts alone is risky; structured, actionable mitigations are needed for reliable security improvements across models and datasets.

Takeaway 7: Prompt design is a first-order factor: across models and datasets, MA-CoT is the only method that consistently reduces vulnerabilities, while Zero-shot and CoT are unreliable and can increase findings relative to Vanilla.

Takeaway 8: The results reveal an interaction between *model behavior* and *task vulnerability mix*. MA-CoT is most effective when dominant weaknesses arise from API configuration and mitigation choices (*e.g.*, cryptography, credential handling), whereas residual findings concentrate on implementation-level hardening (*e.g.*, race conditions, secure wiping). This explains why the same model shows modest gains on one dataset and near elimination on another under the same prompting strategy.

F. Language-Core/Stack Vulnerabilities Drivers

This subsection addresses RQ2 using a strict definition of *language-specificity*. We separate (i) *language-core* drivers (required by the language specification or mandatory runtime semantics) from (ii) *language-stack* drivers, including standard runtime, ecosystem libraries/frameworks, platform/OS APIs, and toolchains. The attribution layers are defined in Table II.

Cross-Language Distribution of Drivers. Table VII summarizes each layer's contribution to attributed weakness instances per language, with primary CWE evidence. The key result is that C security outcomes are shaped mainly by the surrounding stack: OS-level check-then-use filesystem patterns (TOCTOU, CWE-367) and library-level crypto/TLS configuration freedom, with language-core memory/bounds issues forming a

TABLE VII: Layer contributions per language with the dominant CWE signal per layer. Percentages are computed over the attributed weakness instances for each language.

Lang.	Layer Contribution
C	Platform/OS API (38.0%, dominated by CWE-367); Ecosystem crypto/TLS library API (31.2%, dominated by CWE-327 and certificate validation CWEs). Language-core memory and bounds (27.1%, dominated by CWE-119/120). Toolchain/optimization (1.8%, dominated by CWE-14). Remaining cases are sparse or uncategorized.
Java	Application security logic/policy (27.6%, input validation, output encoding, auth configuration). Language-core error model exposure (27.1%, dominated by CWE-600). Standard runtime crypto/TLS surfaces (21.2%, dominated by CWE-327). Language-core secret representation gaps (13.5%, dominated by CWE-259). XML/runtime configuration surfaces and other categories contribute the remainder.
Python	Generation-prone patterns and community idioms (30.8%, *e.g.*, placeholder constants and repetition). Language-core semantics (21.5%, *e.g.*, default-argument timing and string literal behavior relevant to injection). Application security logic/policy (21.5%). Ecosystem library/framework API permissiveness (16.9%, dominated by CWE-327/329 patterns). Runtime/module-state exposure (9.2%).

smaller but important component. Java and Python shift toward exception boundaries, secret representation, configuration surfaces, and permissive APIs. This explains why MA-CoT is strongest in Java and Python, where weaknesses are largely configuration/API-driven, while C retains residual risk from OS/toolchain semantics and language-core memory issues.

C: Separating language-core memory hazards from, library, OS, and toolchain exposures. C shows a split between language-core and language-stack risk. The language-core driver is the lack of intrinsic bounds checking with pervasive raw buffers and pointer arithmetic, consistent with memory-safety CWEs (CWE-119/120) in Table VII. In contrast, several high-frequency issues are not strictly C-specific (Table II): (1) TOCTOU (CWE-367) stems from POSIX check-then-use semantics and can arise in any language using the same OS APIs. (2) Cryptographic and TLS weaknesses are driven by library configuration surfaces (*e.g.*, algorithm/mode selection, verification settings) rather than C syntax. (3) Secure wiping failures (*e.g.*, CWE-14 and related rules) stem from toolchain optimization when non-guaranteed zeroization is used.

C Strengthening Actions by Layer. The mitigation target depends on the layer (Table VII): ① *Language-core (memory and bounds):* Prefer bounded abstractions (length-carrying buffers and checked wrappers), ban unbounded copies, and validate sizes at every boundary. Where feasible, adopt safer subsets (*e.g.*, CERT C / MISRA guidance) and enable sanitizers in testing to expose overflow- and lifetime-related defects early. ② *Platform/OS API (TOCTOU):* Avoid check-then-use flows and prefer atomic, descriptor-based patterns (open-and-check rather than pre-check, fd (file descriptor)-based validation). Avoid separate existence checks and use primitives that reduce race windows (*e.g.*, `openat`, `O_CREAT| O_EXCL`, and `O_NOFOLLOW` where applicable). ③ *Crypto/TLS library layer:* Reduce insecure configuration freedom by using safe wrappers and secure defaults. In MA-CoT, require certificate and hostname verification (no verification bypass) and constrain cryptography to modern AEAD with safe parameter handling. ④ *Toolchain/optimization layer (secure wiping):* Use guaranteed-wipe primitives rather than plain `memset`-based scrubbing that compilers may remove (*e.g.*, `memset_s` where available, or platform-provided secure-zero functions such as `explicit_bzero`/`SecureZeroMemory`).

Takeaway 9: Based on Table II, many high-frequency C findings (*e.g.*, TOCTOU and crypto/TLS misconfiguration) are attributed to platform and library layers rather than the C language-core, while true language-core risk remains concentrated in memory and bounds handling (Table VII).

Java: Security failures concentrate in exception boundaries and string-configured security APIs. Java's managed runtime reduces memory-corruption weaknesses, shifting risk toward configuration-heavy security and error-driven failures. In Table VII, Java shows a strong error-handling contribution dominated by CWE-600, indicating that LLM-generated code often mishandles exceptional control flow (*e.g.*, uncaught exceptions, overly broad catches, error propagation bypassing checks). A second signal is hardcoded secrets (CWE-259), as security-sensitive values are expressed as string literals without a first-class secret type. Cryptographic weaknesses (CWE-327) persist because Java exposes algorithm and mode choices through string-based configuration (*e.g.*, transformation strings, provider selection), making insecure options easy to select without guardrails.

Java Strengthening Actions by Layer. The mitigation goal varies by layer: ① *Language-core error boundaries:* Enforce fail-closed exception handling, avoid catch-all swallowing, ensure security checks and authorization decisions are not bypassed on exceptional paths, and prevent sensitive internal error details from leaking through exception messages. ② *Secret representation and lifetime:* Avoid embedding secrets as string literals or long-lived `Strings` (which cannot be reliably cleared and are easily propagated). Prefer typed secret carriers where feasible (*e.g.*, key objects, keystores, or short-lived `byte[]` buffers with best-effort zeroization) and never log or include secrets in exception text. ③ *Standard runtime crypto/TLS surfaces:* Constrain algorithm selection to modern AEAD by default and require cryptographically secure randomness and standard KDFs for derived keys. For TLS, certificate and hostname verifications are required, and verification-bypass patterns are forbidden. ④ *Application/framework layer:* Use output encoding or safe templating defaults to prevent CWE-79, validate all untrusted inputs to reduce CWE-20, and enforce authentication and password policies (CWE-521) explicitly via configuration and centralized validation.

Python: Expressiveness amplifies configuration risk and parameter reuse. Python's memory safety reduces low-level memory-corruption risk, but cryptographic misuse and input-driven vulnerabilities (*e.g.*, insufficient validation, unsafe string construction) remain common. As shown in Table VII, Python weaknesses stem from permissive APIs and generation-prone idioms, alongside language-core semantics

that make insecure patterns concise (*e.g.*, module-level state, default-argument evaluation reusing values across calls). Models often encode security-critical parameters as copyable constants (*e.g.*, static IVs/nonces, placeholder secrets), and many APIs accept them without enforcing freshness or entropy. This interaction motivates guidance requiring nonce/IV uniqueness, secure randomness, and non-literal secret sourcing.

Python Strengthening Actions by Layer. The following actions target Python-specific risks across language and stack layers. ① *Language-core semantics and state:* Avoid module-level security-critical constants and avoid default-argument traps by requiring per-operation runtime initialization of nonces/IVs/tokens using a CSPRNG. Treat all external inputs as untrusted and apply validation/encoding at security-sensitive sinks (*e.g.*, HTML output, command execution, and query construction). ② *Library/framework APIs:* Prefer high-level cryptographic APIs that enforce randomness and safe modes by construction; explicitly forbid insecure modes and static parameters (*e.g.*, fixed IVs/nonces). For web frameworks, require safe templating/auto-escaping by default and explicit content-type handling.

Takeaway 10: Java and Python weaknesses are characterized by configuration, exception boundaries, and API choices, so MA-CoT is most effective when it constrains these surfaces with secure defaults and fail-closed policies. C retains more residual risk because a larger fraction is driven by OS/toolchain semantics and language-core memory hazards, which require atomic OS patterns and secure-by-construction primitives in addition to prompt guidance.

Takeaway 11: Language-core rules are necessary but insufficient; secure generation must also include stack-level constraints (library defaults, OS-safe patterns, etc.).

VI. Discussion

Our results suggest that LLM-generated code security depends less on "more reasoning" and more on whether prompts provide *actionable* mitigations aligned with dominant weakness types. Across both datasets, MA-CoT was the most secure method, while Zero-shot and CoT were unstable. This stability appears at the severity level: high-severity findings (Blocker plus Critical) drop from 90 to 39 in the primary dataset (56.7%) and from 45 to 2 in LLMSecEval (95.6%).

A key technical signal is that MA-CoT reduces recurring, high-impact CWE classes rather than totals alone. In the primary dataset, C was dominated by CWE-295 and CWE-327, and MA-CoT reduced both (CWE-295=6; CWE-327=2). Remaining issues shift toward implementation-level hardening (*i.e.*, TOCTOU and secure wiping). This is most apparent in C, where the Blocker increase is driven by `c:S5798` rule, flagging insecure-erasure with `memset()` before `free()` due to compiler optimization; the net high-severity burden still decreases (52 to 31). The appendix provides a concrete cross-language illustration of this mechanism for a representative encryption task.

Why generic CoT can increase findings (especially in C). The underperformance of generic CoT is not accidental. CoT expands intermediate design choices, and without concrete mitigation constraints, models often select insecure defaults while producing plausible explanations. In C, this effect is amplified because added planning steps introduce check-then-use file flows and low-level plumbing, where TOCTOU and verification bypass patterns appear. In contrast, MA-CoT narrows these degrees of freedom by prescribing safe primitives and explicit prohibitions, reducing risky configurations.

Layered attribution explains the cross-language gap. The layered attribution in subsection III-F clarifies why improvements differ by language. In C, residual risk is largely driven by language-stack layers, including OS-level filesystem semantics (*e.g.*, TOCTOU) and toolchain behavior (*e.g.*, non-guaranteed wiping), which prompts alone cannot eliminate. In Java and Python, weaknesses are more configuration and API-driven (crypto choices, exception boundaries, secret handling), making them more responsive to MA-CoT constraints that enforce secure defaults and fail-closed behavior.

Dataset shift explains why the same prompting strategy appears moderate in one dataset and nearly eliminated in another. LLMSecEval emphasizes different task families (*i.e.*, Java web patterns), whereas our dataset includes more systems-oriented C scenarios; differences in raw counts reflect task composition and language mix, not evaluation artifacts. Prompt effectiveness should be assessed by *which CWE classes dominate* in a benchmark, not only aggregate totals.

Our study provides three practical and testable insights. ❶ treat prompt design as a security control (MA-CoT is consistently strongest, whereas generic CoT/Zero-shot is not reliable); ❷ report severity and CWE distributions to avoid masking category shifts behind totals; and ❸ view residual MA-CoT findings as hardening opportunities best addressed with secure-by-construction primitives (*i.e.*, guaranteed-wipe routines) rather than expecting the model to consistently implement subtle rules from memory.

VII. Threats to Validity

Construct. We measured security using SAST findings (SonarQube rules), which capture likely weakness patterns but not exploitability. SAST is pattern-sensitive: some rules trigger only for specific idioms. For example, `c:S5798` rule flags `memset()` before `free()` because the write may be optimized away. MA-CoT introduces explicit scrubbing more often, increasing `c:S5798` rule counts even when overall high-severity burden decreases. We interpret such cases as implementation-level hardening signals and document remediation (*e.g.*, guaranteed-wipe primitives). To reduce false positives, two security researchers independently reviewed SonarQube findings and resolved disagreements by consensus.

Internal. LLM outputs vary with decoding, prompts, and provider updates. We mitigated this by applying an identical generation and analysis pipeline across models and prompting methods and using the same analyzer configuration per dataset.

External. The evaluation covered three languages, three models, and two datasets. While the benchmark provides an external check, results may differ for other languages, domains, or task mixes emphasizing different CWE families. Our claims are therefore comparative: under tested conditions, MA-CoT is more reliable than Vanilla, Zero-shot, and CoT.

Conclusion. Aggregated counts can hide variance across tasks and mask category shifts. To improve interpretability, we reported totals alongside severity and CWE distributions and used expert reviews to validate the findings and explain rule-level effects rather than using aggregate metrics alone.

VIII. Conclusion and Future Work

This study evaluates mitigation-aware prompting for secure code generation across three LLMs, three languages (C, Java, Python), and four prompting strategies on a 200-task primary dataset, with external validation on LLMSecEval. The key finding is that mitigation-aware prompting (MA-CoT) yields the most *reliable* security improvements and generalizes under dataset shift. In the primary dataset, high-severity findings (Blocker + Critical) decreased from 90 to 39 (56.7%) across all models and languages. On LLMSecEval, high-severity findings dropped from 45 to 2 (95.6%), and total findings from 73 to 4 (94.5%). These results indicate that: (RQ1) MA-CoT consistently outperforms baselines; (RQ2) improvements hold across languages and models; and (RQ3) residual risk concentrates in narrower hardening-oriented CWE patterns.

A practical contribution of this work is the distinction between language-core and language-stack causes, which prevents overstatement of what is truly "language-specific." This distinction is critical for mitigation: language-core issues demand safer constructs and bounded memory handling, whereas stack-driven issues require secure-by-default libraries, OS-atomic file patterns, and toolchain-safe primitives. Positioning mitigation at the appropriate layer transforms cross-language comparisons into actionable guidance, enhancing system *reliability* rather than yielding broad claims.

Beyond aggregate reductions, the results clarify *what changes*. MA-CoT removes recurring high-risk patterns while shifting residual issues toward implementation-level hardening. In C, the net high-severity burden decreases (52 to 31) even though Blocker increases due to strict secure-erasure enforcement (dominated by `c:S5798` rule), which flags non-guaranteed wiping with `memset`. At the model level, Table VI shows that responsiveness to mitigation guidance varies (*i.e.*,`claude-4.5` benefits strongly, while `gpt-5` is more stable on the primary dataset), reinforcing that model selection and prompting policy should be evaluated jointly for dependable code generation.

For researchers, the main contribution is evidence that prompt design is a first-order *reliability control*: generic prompting and unguided reasoning are inconsistent, whereas mitigation-aware constraints yield consistent reductions across datasets. Future work should extend evaluation beyond static analysis by combining static and dynamic validation and standardizing secure-by-construction primitives and wrappers (*e.g.*, guaranteed wiping, safe file-open patterns) to eliminate residual hardening-oriented failure modes under MA-CoT.

References

[1] M. Chen, J. Tworek, H. Jun, Q. Yuan, H. de Oliveira Pinto, J. Kaplan, H. Edwards, Y. Burda, N. Joseph, G. Brockman *et al.*, "Evaluating large language models trained on code," *arXiv preprint arXiv:2107.03374*, 2021.

[2] J. Lin and D. Mohaisen, "From large to mammoth: A comparative evaluation of large language models in vulnerability detection," in *32nd Annual Network and Distributed System Security Symposium, NDSS 2025, San Diego, California, USA, February 24-28, 2025*. The Internet Society, 2025.

[3] O. Asare, M. Nagappan, and N. Asokan, "A user-centered security evaluation of copilot," in *ICSE*, 2024.

[4] M. B. Messaoud, A. Miladi, I. Jenhani, M. W. Mkaouer, and L. Ghadhab, "Duplicate bug report detection using an attention-based neural language model," *IEEE Trans. Reliab.*, vol. 72, no. 2, pp. 846–858, 2023. [Online]. Available: https://doi.org/10.1109/TR.2022.3193645

[5] I. Medeiros, N. Neves, and M. Correia, "Statically detecting vulnerabilities by processing programming languages as natural languages," *IEEE Trans. Reliab.*, vol. 71, no. 2, pp. 1033–1056, 2022. [Online]. Available: https://doi.org/10.1109/TR.2021.3137314

[6] N. O. Jaffal, M. Alkhanafseh, and D. Mohaisen, "Large language models in cybersecurity: A survey of applications, vulnerabilities, and defense techniques," *AI*, vol. 6, no. 9, 2025. [Online]. Available: https://www.mdpi.com/2673-2688/6/9/216

[7] —, "Adoption of developer ai tools (copilot, etc) 2022 - 2025: Data & graphs show increase in ai use as developers evolve - gitclear," 12 2025, [Online; accessed 2025-12-27]. [Online]. Available: https://www.gitclear.com/research/developer_ai_assistant_adoption_by_year_with_ai_delegation_buckets

[8] B. Yetistiren, I. Özsoy, M. Ayerdem, and E. Tüzün, "Evaluating the code quality of ai-assisted code generation tools: An empirical study on github copilot, amazon codewhisperer, and chatgpt," *CoRR*, vol. abs/2304.10778, 2023.

[9] H. Pearce, B. Ahmad, B. Tan, B. Dolan-Gavitt, and R. Karri, "Asleep at the keyboard? assessing the security of github copilot's code contributions," in *IEEE Symp. on Security & Privacy*, 2022.

[10] Y. Fu, P. Liang, A. Tahir, Z. Li, M. Shahin, J. Yu, and J. Chen, "Security weaknesses of copilot-generated code in github projects: An empirical study," *CoRR*, vol. abs/2310.02059, 2025.

[11] G. Sandoval, H. Pearce, T. Nys, R. Karri, S. Garg, and B. Dolan-Gavitt, "Lost at c: A user study on the security implications of large language model code assistants," in *Proceedings of the 32nd USENIX Security Symposium*, 2023, pp. 2205–2222.

[12] O. Asare, M. Nagappan, and N. Asokan, "Is github's copilot as bad as humans at introducing vulnerabilities in code?" *Empir. Softw. Eng.*, vol. 28, no. 6, p. 129, 2023. [Online]. Available: https://doi.org/10.1007/s10664-023-10380-1

[13] S. Ullah, M. Han, S. Pujar, H. Pearce, A. Coskun, and G. Stringhini, "Llms cannot reliably identify and reason about security vulnerabilities (yet?): A comprehensive evaluation, framework, and benchmarks," in *IEEE Symposium on Security and Privacy*, 2024.

[14] J. Wei, X. Wang, D. Schuurmans, M. Bosma, B. Ichter, F. Xia, E. H. Chi, Q. V. Le, and D. Zhou, "Chain-of-thought prompting elicits reasoning in large language models," in *Proceedings of the 36th Conference on Neural Information Processing Systems (NeurIPS)*, 2022.

[15] T. Kojima, S. S. Gu, M. Reid, Y. Matsuo, and Y. Iwasawa, "Large language models are zero-shot reasoners," in *Advances in Neural Information Processing Systems 35: Annual Conference on Neural Information Processing Systems 2022, NeurIPS 2022, New Orleans, LA, USA, November 28 - December 9, 2022*, S. Koyejo, S. Mohamed, A. Agarwal, D. Belgrave, K. Cho, and A. Oh, Eds., 2022. [Online]. Available: http://papers.nips.cc/paper_files/paper/2022/hash/8bb0d291acd4acf06ef112099c16f326-Abstract-Conference.html

[16] C. Tony, N. E. D. Ferreyra, M. Mutas, S. Dhif, and R. Scandariato, "Prompting techniques for secure code generation: A systematic investigation," *ACM TOSEM*, vol. 34, no. 8, pp. 225:1–225:53, 2025. [Online]. Available: https://doi.org/10.1145/3722108

[17] M. Kharma, A. Sabbah, M. AlKhanafseh, M. Hammoudeh, and D. Mohaisen, "An empirical evaluation of llm-generated code security across prompting methods," 2026, submitted to Empirical Software Engineering.

[18] C. An, J. Zhang, M. Zhong, L. Li, S. Gong, Y. Luo, J. Xu, and L. Kong, "Why does the effective context length of llms fall short?" *CoRR*, vol. abs/2410.18745, 2024.

[19] S.-C. Dai, J. Xu, and G. Tao, "Rethinking the evaluation of secure code generation," 2025. [Online]. Available: https://arxiv.org/abs/2503.15554

[20] D. Cotroneo, R. D. Luca, and P. Liguori, "Devaic: A tool for security assessment of ai-generated code," *Inf. Softw. Technol.*, vol. 177, p. 107572, 2025. [Online]. Available: https://doi.org/10.1016/j.infsof.2024.107572

[21] B. Lin, S. Wang, Y. Qin, L. Chen, and X. Mao, "Give llms a security course: Securing retrieval-augmented code generation via

knowledge injection," in *CCS*, C. Huang, J. Chen, S. Shieh, D. Lie, and V. Cortier, Eds. ACM, 2025, pp. 3356–3370. [Online]. Available: https://doi.org/10.1145/3719027.3765049
[22] N. Huynh and B. Lin, "Large language models for code generation: A comprehensive survey of challenges, techniques, evaluation, and applications," *CoRR*, vol. abs/2503.01245, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2503.01245
[23] J. H. Klemmer, S. A. Horstmann, N. Patnaik, C. Ludden, C. B. Jr., C. Powers, F. Massacci, A. Rahman, D. Votipka, H. R. Lipford, A. Rashid, A. Naiakshina, and S. Fahl, "Using AI assistants in software development: A qualitative study on security practices and concerns," in *CCS*. ACM, 2024, pp. 2726–2740. [Online]. Available: https://doi.org/10.1145/3658644.3690283
[24] N. Tihanyi, T. Bisztray, M. A. Ferrag, R. Jain, and L. C. Cordeiro, "How secure is ai-generated code: a large-scale comparison of large language models," *Empir. Softw. Eng.*, vol. 30, no. 2, p. 47, 2025. [Online]. Available: https://doi.org/10.1007/s10664-024-10590-1
[25] P. Sahoo, A. K. Singh, S. Saha, V. Jain, S. Mondal, and A. Chadha, "A systematic survey of prompt engineering in large language models: Techniques and applications," *CoRR*, vol. abs/2402.07927, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2402.07927
[26] B. Chen, Z. Zhang, N. Langrené, and S. Zhu, "Unleashing the potential of prompt engineering in large language models: a comprehensive review," *CoRR*, vol. abs/2310.14735, 2023. [Online]. Available: https://doi.org/10.48550/arXiv.2310.14735
[27] E. Basic and A. Giaretta, "Large language models and code security: A systematic literature review," *CoRR*, vol. abs/2412.15004, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2412.15004
[28] A. Schaad, S. Götz, and D. Binder, "You still have to study on the security of LLM generated code," in *ICT Systems Security and Privacy Protection - 40th IFIP International Conference, SEC 2025, Maribor, Slovenia, May 21-23, 2025, Proceedings, Part II*, ser. IFIP Advances in Information and Communication Technology, L. N. Zlatolas, K. Rannenberg, T. Welzer, and J. García-Alfaro, Eds., vol. 746. Springer, 2025, pp. 111–124. [Online]. Available: https://doi.org/10.1007/978-3-031-92886-4_8
[29] A. Datta, A. Aljohani, and H. Do, "Secure code generation at scale with reflexion," *CoRR*, vol. abs/2511.03898, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2511.03898
[30] G. Kim, P. Baldi, and S. McAleer, "Language models can solve computer tasks," in *Proceedings of the 37th Conference on Neural Information Processing Systems (NeurIPS)*, 2023.
[31] J. Li, F. Rabbi, B. Yang, S. Wang, and J. Yang, "Secure-instruct: An automated pipeline for synthesizing instruction-tuning datasets using llms for secure code generation," 2025. [Online]. Available: https://arxiv.org/abs/2510.07189
[32] J. Shi and T. Zhang, "RESCUE: retrieval augmented secure code generation," *CoRR*, vol. abs/2510.18204, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2510.18204
[33] J. Zhao, Y. Sun, C. Huang, C. Liu, Y. Guan, Y. Zeng, and Y. Liu, "Towards secure code generation with llms: A study on common weakness enumeration," *IEEE Transactions on Software Engineering*, vol. 51, no. 12, pp. 3507–3523, 2025.
[34] M. Nazzal, I. Khalil, A. Khreishah, and N. Phan, "Promsec: Prompt optimization for secure generation of functional source code with large language models (llms)," in *CCS*. ACM, 2024, pp. 2266–2280. [Online]. Available: https://doi.org/10.1145/3658644.3690298
[35] Y. Nong, M. Aldeen, L. Cheng, H. Hu, F. Chen, and H. Cai, "Chain-of-thought prompting of large language models for discovering and fixing software vulnerabilities," *CoRR*, vol. abs/2402.17230, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2402.17230
[36] X. Jiang, Y. Dong, L. Wang, Z. Fang, Q. Shang, G. Li, Z. Jin, and W. Jiao, "Self-planning code generation with large language models," *ACM Trans. Softw. Eng. Methodol.*, vol. 33, no. 7, pp. 182:1–182:30, 2024. [Online]. Available: https://doi.org/10.1145/3672456
[37] M. F. Kharma, S. Choi, M. Alkhanafseh, and D. Mohaisen, "Security and quality in llm-generated code: a multi-language, multi-model analysis," *IEEE Transactions on Dependable and Secure Computing*, no. 01, pp. 1–15, 2026. [Online]. Available: https://doi.org/10.1109/TDSC.2026.3672745
[38] M. U. Shahid, C. M. Ahmed, and R. Ranjan, "Llm-csec: Empirical evaluation of security in c/c++ code generated by large language models," 2025. [Online]. Available: https://arxiv.org/abs/2511.18966
[39] J. Peng, L. Cui, K. Huang, J. Yang, and B. Ray, "Cweval: Outcome-driven evaluation on functionality and security of LLM code generation," in *IEEE/ACM International Workshop on Large Language Models for Code, LLM4Code@ICSE 2025, Ottawa, ON, Canada, May 3, 2025*. IEEE, 2025, pp. 33–40. [Online]. Available: https://doi.org/10.1109/LLM4Code66737.2025.00009
[40] M. Vero, N. Mündler, V. Chibotaru, V. Raychev, M. Baader, N. Jovanovic, J. He, and M. T. Vechev, "Baxbench: Can llms generate correct and secure backends?" in *ICML*. OpenReview.net, 2025. [Online]. Available: https://openreview.net/forum?id=il3KRr4H9u
[41] J. Chen, H. Huang, Y. Lyu, J. An, J. Shi, C. Yang, T. Zhang, H. Tian, Y. Li, Z. Li, X. Zhou, X. Hu, and D. Lo, "Secureagentbench: Benchmarking secure code generation under realistic vulnerability scenarios," *CoRR*, vol. abs/2509.22097, 2025. [Online]. Available: https://doi.org/10.48550/arXiv.2509.22097
[42] —, "Gpt-5 model | openai api," https://platform.openai.com/docs/models/gpt-5, 09 2025, (Accessed on 2025-09-24).
[43] ——, "What's new in claude 4.5 - claude doc," https://platform.claude.com/docs/en/about-claude/models/whats-new-claude-4-5, 09 2025, (Accessed on 2025-09-24).
[44] ——, "Gemini models | gemini api | google ai for developers," https://ai.google.dev/gemini-api/docs/models, 09 2025, (Accessed on 2025-09-24).
[45] ——, "Code quality, security & static analysis tool with SonarQube," https://www.sonarsource.com/products/sonarqube/, 05 2024, (Accessed on 05/12/2024).
[46] C. Tony, M. Mutas, N. E. D. Ferreyra, and R. Scandariato, "Llmseceval: A dataset of natural language prompts for security evaluations," in *MSR*. IEEE, 2023, pp. 588–592. [Online]. Available: https://doi.org/10.1109/MSR59073.2023.00084
[47] M. Bruni, F. Gabrielli, M. Ghafari, and M. Kropp, "Benchmarking prompt engineering techniques for secure code generation with GPT models," in *Forge@ICSE*. IEEE, 2025, pp. 93–103. [Online]. Available: https://doi.org/10.1109/Forge66646.2025.00018
[48] M. L. Siddiq, J. C. da Silva Santos, S. Devareddy, and A. Muller, "SALLM: security assessment of generated code," in *ASE Workshops*. ACM, 2024, pp. 54–65. [Online]. Available: https://doi.org/10.1145/3691621.3694934
[49] S. Liu, B. Sabir, S. I. Jang, Y. Kansal, Y. Gao, K. Moore, A. Abuadbba, and S. Nepal, "From solitary directives to interactive encouragement! LLM secure code generation by natural language prompting," *CoRR*, vol. abs/2410.14321, 2024. [Online]. Available: https://doi.org/10.48550/arXiv.2410.14321

APPENDIX A
EXAMPLE ENTRIES FROM THE MITIGATION-AWARE DATASET

This appendix provides two representative records from the Mitigation-Aware dataset. Each entry connects a category to its mitigation rules, language-specific guidance, code review checklist, and fine-tuning examples. The format shows how the dataset structures both general and implementation-level secure-coding practices.

A. Example 1: Language Basics

Scope. Baseline security and quality rules.
Applicable Languages. Language-agnostic.
General Rules.

- Default to secure coding patterns. Treat all inputs as untrusted and fail closed with clear error messages.
- Validate type, length, range, and encoding. Sanitize or reject early. Normalize paths and prevent traversal.
- Never hard code passwords or secrets. Use configuration files or a secret manager.
- Avoid dynamic code loading and insecure deserialization.
- Use trusted cryptographic libraries. Prefer AEAD (AES-GCM or ChaCha20-Poly1305). Derive keys with Argon2id, scrypt, or PBKDF2-HMAC-SHA-256 using unique salts.
- Enforce TLS 1.2 or TLS 1.3 with certificate and hostname verification.

- Use safe file operations, atomic writes, and restrictive permissions.
- Maintain memory safety by checking allocation results, bounding copies, ensuring null termination, and freeing memory exactly once.
- Zeroize sensitive data with a reliable API before freeing memory.
- Never log secrets, tokens, or plaintext. Use constant format strings and redact sensitive data.
- Prevent internal error details from leaking to external users.
- Use atomic operations and thread-safe APIs to prevent race conditions.
- Enable compiler hardening flags, run static analysis, and verify dependencies.
- Limit network activity to secure protocols and minimal exposure.

Language-Specific Guidance.

C - avoid `gets`, `strcpy`, `sprintf`, and unbounded `scanf`. Use `fgets` and `snprintf`, check return values, use one cleanup label, and clear secrets with `memset_s`. Enforce TLS verification when using libcurl.

C++ - prefer `std::string`, `std::vector`, RAII, and smart pointers. Use `std::span` for safe access and avoid `std::rand` for security.

Python - use context managers and type hints. Prefer the `secrets` module and the `cryptography` library. Avoid `eval` and unsafe `pickle`. Use `ssl.create_default_context` with TLS 1.2+ and enable certificate checks.

Java - use `StandardCharsets.UTF_8` and try-with-resources. Apply safe deserialization patterns. For encryption, use `AES/GCM /NoPadding` with `GCMParameterSpec(128, iv)` and derive keys via `PBKDF2WithHmacSHA256`. Never disable certificate validation.

Code Review Checklist. Inputs validated; no hard coded secrets; no unsafe reflection or insecure deserialization; modern KDFs and AEAD in crypto; TLS verification enforced; safe file operations; logs redact sensitive data; no internal error leakage; memory and overflow checks present; concurrency hazards mitigated; build hardening flags and testing active.

Fine-tuning examples. Generic - respond to any request to hard code a password by showing secure credential handling with environment variables or a secret manager.

Python -

```
token = random.random()
requests.get(url, verify=false)
```

Secure alternative: use `secrets.token_urlsafe` for randomness and a verified SSL context enforcing TLS 1.2 or higher.

Java - authenticated encryption with proper IV handling. Use AES/GCM with a 12-byte random IV from `SecureRandom`, paired with `GCMParameterSpec(128, iv)`, and keys derived via PBKDF2-HMAC-SHA-256 with a unique salt.

B. Example 2: Cryptography

Scope. Encryption, hashing, key management, and TLS usage.
Applicable Languages. Language-agnostic.
Additional Tag. Data safety and security.
General Rules.

- Use vetted cryptographic libraries and modern algorithms. Never implement cryptography manually.
- Ban MD5, SHA-1, DES/3DES, RC4, and ECB mode. Use AEAD encryption (AES-GCM/ChaCha20-Poly1305).
- Derive keys with Argon2id, scrypt, or PBKDF2-HMAC-SHA-256 and a random salt.
- Use secure random generators for keys, salts, and nonces. For RSA, prefer OAEP ($\geq$2048 bits) or modern curves (X25519, Ed25519).
- Enforce TLS 1.2 or TLS 1.3 and never disable certificate validation.

Language-Specific Guidance. C - use OpenSSL EVP, libsodium, or Botan for encryption, KDFs, and random generation. When using libcurl:

```
curl_easy_setopt(curl,
        CURLOPT_SSL_VERIFYPEER, 1L);
curl_easy_setopt(curl,
        CURLOPT_SSL_VERIFYHOST, 2L);
curl_easy_setopt(curl, CURLOPT_SSLVERSION,
            CURL_SSLVERSION_TLSv1_2
            | CURL_SSLVERSION_MAX_TLSv1_3);
```

Check return values, use bounds-checked buffers, clear key material with `memset_s`, and never disable certificate checks.

C++ - use vetted cryptographic libraries. Ensure HTTPS/TLS clients enforce TLS 1.2+ with hostname verification and avoid deprecated cipher suites.

Python - use the `cryptography` library. Enforce TLS 1.2 or higher:

```
import ssl
ctx = ssl.create_default_context()
ctx.check_hostname = True
ctx.verify_mode = ssl.CERT_REQUIRED
if hasattr(ssl, "TLSVersion"):
    ctx.minimum_version = ssl.TLSVersion.
        TLSv1_2
else:
    ctx.options |= ssl.OP_NO_SSLv2 | ssl.
        OP_NO_SSLv3
    ctx.options |= ssl.OP_NO_TLSv1 | ssl.
        OP_NO_TLSv1_1
```

Java - use `Cipher("AES/GCM/NoPadding")` with `GCMParame -terSpec(128, iv)`. Generate IVs and salts with `SecureRandom`, derive keys using `PBKDF2WithHmacSHA256`, use try-with-resources, and enforce TLS 1.2 or higher.

APPENDIX B
INTERPRETING `c:S5798` RULE IN MA-COT C OUTPUTS

What `c:S5798` rule Reports. `c:S5798` rule (*"`memset` should not be used to delete sensitive data"*) flags cases where a program attempts to scrub a buffer using `memset()`

immediately before destroying it (*i.e.*, via `free()`). The warning reflects that such writes may be removed by compiler optimization, so the intended zeroization is not guaranteed.

Why Vanilla Can Appear "cleaner". Vanilla triggered `c:S5798` rule zero times in our reports because it typically does not introduce explicit buffer-scrubbing calls (*i.e.*, `memset()` before `free()`), which are the specific patterns targeted by the rule. The absence of `c:S5798` rule findings should not be interpreted as a guarantee of secure data lifetime handling, as it only indicates that the insecure-erasure pattern is not present.

Actionable Remediation. When explicit zeroization is required, `memset()` should be replaced with a guaranteed-wipe primitive (*i.e.*, `memset_s` where available, or an equivalent secure-zero routine) so that scrubbing cannot be optimized away. This removes `c:S5798` rule while preserving the intended defense, and explains why MA-CoT can reduce high-severity findings while shifting a small subset of residual issues into Blocker due to strict secure-erasure rules.

APPENDIX C
CROSS-PROMPT AND CROSS-LANGUAGE CODE SNIPPETS FOR ONE TASK EXAMPLE

This appendix provides a qualitative case study to complement the aggregate results in section V. We select one representative task from the primary dataset (Task 79: *Encrypt a Message Using Secret Key*, associated with CWE-327) and show code snippets generated by `claude-4.5` across all prompting methods (Vanilla, ZeroShot, CoT, and MA-CoT) and three programming languages (Java, Python, and C). The goal is to illustrate how prompt design changes concrete security-relevant implementation choices (e.g., cipher mode, key derivation, IV handling, and cleanup behavior), which aligns with the CWE-level findings.

A. Task and Setup

Task. Task 79 asks the model to create a cipher and encrypt a message using a secret key.

Scope. All snippets referenced in this appendix were generated by `claude-4.5` for the same task under the four prompting methods evaluated in this paper. The snippets are reproduced as generated and are shown as excerpts to highlight security-relevant implementation choices. To preserve space, the full code snippets are available at GitHub.

B. Takeaways

The analysis provided in this section mirrors the aggregate CWE findings in section V-D: Vanilla and ZeroShot often preserve insecure cryptographic defaults (*e.g.*, ECB), CoT improves structure but may still use non-AEAD modes (*e.g.*, CBC), and MA-CoT consistently shifts implementations toward AEAD-based designs with stronger parameter handling.

The change under MA-CoT is not only stylistic. It alters implementation choices that directly affect security (cipher mode, Key Derivation Function (KDF) usage, random salt/IV generation, validation, and secure cleanup), which explains the observed reductions in recurring cryptographic weakness patterns.